

CmpE 260 — Interpreter Project

Melang Language Design Report

Mert Yilmaz, 2024XXXXXX

Metehan Yilmaz, 2024XXXXXX

May 2026

1 Introduction

Melang is a small interpreted language built for this project. The name comes from the two authors' first names (*Mert* and *Metehan*) combined with *lang*. The interpreter follows the standard pipeline: a hand-written lexer produces tokens, a recursive descent parser builds an AST, and a tree-walking evaluator executes the program while maintaining an explicit environment.

Melang supports integers, booleans, strings, and lists; first-class functions with closures; recursive definitions; and both static and dynamic scoping selectable at runtime.

2 Scoping Decision

Melang implements **static (lexical) scoping** as the default. Dynamic scoping is also available via the `--scope dynamic` flag (bonus B4).

Under static scoping, a function resolves free variables in the environment where it was *defined*. Under dynamic scoping, they are resolved where the function is *called*.

Why static scoping

Static scoping makes programs easier to reason about. Given any function definition in the source code, it is possible to determine exactly which variables it can see without knowing how or from where the function will be called. Dynamic scoping requires tracking the full call chain at runtime to understand variable resolution, which makes programs harder to follow and debug.

Most practical languages (Python, Java, C, ML) use static scoping for this reason. Dynamic scoping survives mainly in older Lisp dialects and in shell scripting, where the convenience of implicit argument passing outweighs the readability cost.

Concrete example where the two strategies differ

```
let x = 1;

let f = fun() -> x end;

let call_with_x = fun(newx) ->
  let x = newx;
  f()
end;

print(call_with_x(99));
```

- **Static scoping:** `f` was defined where `x = 1`, so `f()` returns 1. Output: 1.
- **Dynamic scoping:** `f` is called inside `call_with_x`, where `x = 99`. Output: 99.

Running the interpreter with `--scope static` and `--scope dynamic` on the same source produces these different outputs.

3 Evaluation Strategy

Melang uses **call-by-value** (strict evaluation). Before a function is called, all argument expressions are fully evaluated to values. Those values are then bound to the function's parameters in the new environment frame.

Call-by-value vs. call-by-name

Under **call-by-name**, argument expressions are passed unevaluated. Each time the parameter is used inside the function body, the argument expression is re-evaluated from scratch.

The difference matters when an argument has a side effect. Consider:

```
let counter = 0;

let increment = fun() ->
  counter = counter + 1;
  counter
end;

let use_twice = fun(v) ->
  v + v
end;

print(use_twice(increment()));
```

- **Call-by-value:** `increment()` is evaluated once before the call. `counter` becomes 1, and `v = 1`. `v + v = 2`. Output: 2.
- **Call-by-name:** `increment()` would be re-evaluated each time `v` appears. First use: `counter` becomes 1, produces 1. Second use: `counter` becomes 2, produces 2. `1 + 2 = 3`. Output: 3.

Melang implements call-by-value: the evaluator processes each argument node to a value before binding.

4 Closure Representation

A closure in Melang is an instance of the `Closure` class, which stores three things:

- **params** — the list of parameter name strings
- **body** — the body AST (Block node)
- **env** — a reference to the environment frame that was active when the `fun` expression was evaluated

When `fun(x) -> x + 1 end` is evaluated, the interpreter creates a `Closure` object and stores a pointer to the current environment. Later, when the closure is called, a new frame is created whose parent is that stored environment (for static scoping) or the call-site environment (for dynamic scoping).

The key detail is that `env` is a reference, not a copy. This means a closure and its surrounding code share the same frame object. Mutations made through the closure (via assignment) are visible in the outer scope, and vice versa. This is what makes the counter example work:

```

let make_counter = fun(start) ->
  let count = start;
  fun() ->
    count = count + 1;
    count
  end
end;

let c = make_counter(0);
print(c()); (* 1 *)
print(c()); (* 2 *)

```

The inner `fun` captures the frame containing `count`. Each call to `c()` modifies `count` in that shared frame, so the value persists between calls.

5 Environment Management

An environment is a plain Python dictionary with two keys: `bindings` (a `str -> value` map) and `parent` (a reference to the enclosing frame, or `None` for the global frame).

Three operations are defined:

- **extend(name, value, env)** — adds a new binding in the current frame. Used for `let` and for parameter binding when a function is called.
- **lookup(name, env)** — searches the chain from the current frame upward until the name is found. Raises a runtime error if not found.
- **update(name, value, env)** — walks up the chain to find the nearest frame where the name is already bound, then modifies it there. Used for assignment (`x = expr`). Raises a runtime error if the name is unbound anywhere in the chain.

Only function calls create new frames. `if` branches and `while` bodies do not introduce new scopes — a `let` inside an `if` branch binds in the enclosing frame.

Required tracing exercise

The following program is traced under static (lexical) scoping:

```

let x = 1;
let f = fun(a) ->
  let y = a + x;
  fun() ->
    x = x + 1;
    y + x
  end
end;
let g = f(10);
print(g());
print(g());

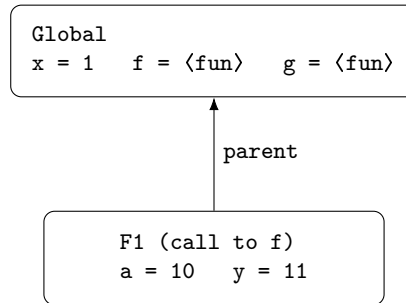
```

Step 1 — Global frame after the first three `let` statements.

Global
x = 1 f = <fun> g = ?

Step 2 — Evaluating $f(10)$.

A new frame **F1** is created for the call. Its parent is **Global** (because **f** was defined there — static scoping). **a = 10** is bound in **F1**. Then `let y = a + x` evaluates $10 + 1 = 11$ and binds **y = 11** in **F1**. The inner `fun()` captures **F1** as its defining environment. This closure is returned and bound to **g** in **Global**.



Step 3 — First call to `g()`.

A new frame **F2** is created. Its parent is **F1** (the closure's captured environment).

- **x = x + 1**: lookup of **x** walks **F2** → **F1** → **Global**, finds **x = 1**. The *update* operation also walks up and modifies **x** in **Global**. Now **x = 2** in **Global**.
- **y + x**: lookup of **y** finds it in **F1** (**y = 11**). Lookup of **x** finds it in **Global** (**x = 2**). Result: $11 + 2 = 13$.

Output of first `print(g())`: **13**.

Step 4 — Second call to `g()`.

A new frame **F3** is created with parent **F1**.

- **x = x + 1**: **x** is looked up in **Global** where it is now 2. It is updated to 3 in **Global**.
- **y + x**: **y** is found in **F1** (**y = 11**, unchanged). **x** is found in **Global** (**x = 3**). Result: $11 + 3 = 14$.

Output of second `print(g())`: **14**.

Why does **y not change between calls?** **y** was bound in **F1** when `f(10)` was called, and nothing ever assigns to it again. The assignment inside `g` targets **x**, not **y**. Since **x** lives in **Global** and **y** lives in **F1**, they are in different frames and the mutation of **x** does not affect **y**.

Which frame is modified by **x = x + 1?** The *update* operation starts at the current call frame and walks up. **x** is not bound in **F2** (or **F3**), not in **F1**, but is found in **Global**. So **Global** is the frame that gets modified.

6 Challenges

Recursive bindings. The most non-obvious part of the implementation was making `let factorial = fun(n) -> ... factorial(n-1) ... end` work. When the `fun` expression is evaluated, the interpreter needs `factorial` to already be in scope so the closure can capture it. But `factorial` is not bound yet at that point.

The solution is to bind the name to **None** as a placeholder first, then evaluate the `fun` expression (which captures the environment containing the placeholder), and finally overwrite the binding with the resulting closure. Because the closure holds a reference to the environment dictionary (not a copy), the overwrite is visible inside the closure.

Assignment vs. let semantics. Getting the distinction between `let x = expr` (new binding in current frame) and `x = expr` (update existing binding up the chain) right took some care, especially for closures that mutate captured variables. The counter example depends entirely on this: `count = count + 1` inside the inner function must find and modify `count` in the outer frame, not create a new local binding.

Boolean before integer in type checks. In Python, `bool` is a subclass of `int`, so `isinstance(True, int)` returns `True`. Every type check in the evaluator tests for `bool` before `int`; reversing the order would silently treat booleans as integers.

Integer division truncation direction. Python's `//` operator truncates toward negative infinity, but the spec requires truncation toward zero (like C and Java). For example, $-7/2$ should be -3 , not -4 . The fix is to use `math.trunc(a / b)` instead of `a // b`.

7 Grammar Ambiguity

Two potential ambiguities arose during the design of the grammar.

Assignment vs. equality. The token sequence `x = ...` could be the start of an assignment statement (`x = expr`) or an expression where `x` is compared with `==`. This is resolved at the lexer level: `=` and `==` are two distinct token types. The parser checks whether the current token is an identifier and the next token is `ASSIGN` (single `=`) to decide whether to parse an assignment. If the next token is `EQ` (`==`), the expression falls through to the comparison rule.

Dangling else. In many languages, `if A then if B then x else y` is ambiguous because it is unclear whether the `else` closes the inner or the outer `if`. Melang avoids this completely because every `if` block is explicitly closed by `end`. The grammar rule is `if expr then block else block end`, so there is no ambiguity about which `if` an `else` belongs to.